

LLM Agents, Tools, and Controlled Execution

An agent chooses actions within an application-defined environment. Reliable operation requires more than a model loop: tools need schemas, authorization, bounded execution, durable state, and safe handling of ambiguous outcomes. External content is data, not authority to expand the user's request.

For: Engineers considering tool-using assistants who need to reason about permission boundaries, state, budgets, approvals, and recovery from partially completed actions.

Suggested pace: 3 weeks using mocked tools | APIs and LLM application basics

What you will be able to do

- Decide when a deterministic workflow is preferable to an agent.
- Define narrow tool contracts, permissions, and input/output validation.
- Control cost, recursion, retries, and high-impact actions.
- Recover from crashes without blindly repeating external side effects.

Build the right mental model

Planning is not authorization

A model proposing an action does not grant permission to perform it. The application must check user intent, tool scope, resource access, and any required approval. A retrieved document or tool result cannot override those boundaries by containing instructions.

State must be durable where effects matter

A conversation transcript alone may not be a reliable operation log. Persist stable operation IDs, status, and checkpoints around side effects. A crash after a provider acted but before local recording creates an ambiguous outcome that needs reconciliation.

Bound the search and execution

Set limits on steps, tokens, wall time, retries, and resource use. Validate tool arguments and outputs. Use safe defaults and a terminal failure path instead of allowing an agent to keep trying increasingly broad actions until something appears to work.

Study with evidence.

Use mocked tools and synthetic data first. Separate read-only tools from state-changing tools and require explicit approval for consequential actions. Test adversarial-looking text as data without allowing it to trigger real communication, deletion, or purchases.

Week 1: Task and tool design

Choose a narrow task and compare a deterministic pipeline with agent planning. Define tool schemas, identity scope, allowed resources, and validation. Start with read-only tools and clear completion criteria.

Deliverable and check: A task contract and tool inventory with explicit permissions and non-goals.

Week 2, first half: Budgets and evaluation

Add step, time, and cost limits, structured outcomes, and a small test set. Include missing information, invalid tool output, and conflicting retrieved instructions. Measure task completion and unsafe-action attempts separately.

Deliverable and check: An evaluation report with successful, failed, and appropriately refused or escalated cases.

Week 2, second half: Durable effects

Introduce one mocked state-changing tool with stable operation identity and approval. Record planned, submitted, confirmed, and uncertain states. Simulate worker crashes around the external call.

Deliverable and check: A recovery path that queries or reconciles status before repeating an uncertain action.

Week 3: Operational review

Inspect logs for sensitive content, review permissions, and define support escalation. Test cancellation, timeout, repeated tool failure, and budget exhaustion. Keep human-visible confirmation for important outcomes.

Deliverable and check: A bounded workflow with an auditable action history and a clear stop condition.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Resume after a tool completed but the worker crashed

An assistant is authorized to create one support ticket through a provider. The provider creates the ticket, but the worker crashes before saving the returned ticket ID. A naive restart could create a duplicate.

A worked approach

1. Before the call, persist a stable operation ID, validated payload, authorization context, and submitted state. Pass a provider idempotency key if supported, and do not generate a fresh identity on retry.
2. On recovery, inspect the durable operation. Query provider status or retry under the same supported idempotency contract. If the outcome cannot be established safely, mark it uncertain and escalate instead of silently creating another ticket.
3. After confirmation, store the provider result and checkpoint completion. Test duplicate callbacks, cancelled user requests, and a different payload under the same operation ID. Keep unrelated retrieved text from altering the authorized destination or action.

Expected result

The workflow can explain what was requested, authorized, attempted, and confirmed. Recovery preserves the logical action identity rather than trusting the model to remember whether something happened.

Verify the result

- Is operation identity durable before the side effect?
- Can an uncertain outcome trigger an uncontrolled duplicate?
- Are external instructions unable to expand tool permissions?

Extend the exercise

Add a high-impact action that requires human approval. Bind approval to the exact action and parameters, expire it appropriately, and require a new approval if the proposed action materially changes.

Make the learning stick.

When should I avoid an agent?

When a deterministic workflow solves the task reliably or when actions cannot be constrained and evaluated adequately. Dynamic planning adds flexibility but also uncertainty, latency, cost, and recovery complexity.

Can a system prompt enforce all safety boundaries?

No. Prompts can guide behavior, but authorization, sandboxing, tool validation, and resource controls belong in the application and infrastructure. Never give the model broader authority than the task requires and rely only on wording to contain it.

Does retrying improve reliability automatically?

Only for appropriately classified failures under a bounded policy. Repeating an irreversible or uncertain side effect can worsen the outcome. Use stable identities, status reconciliation, and escalation where needed.

Avoid these traps

- Treating retrieved text as permission to act.
- Using an unbounded reasoning/tool loop as a recovery strategy.
- Repeating external effects after a crash without reconciliation.

Ready for the next step?

- Tool permissions and user authorization are explicit.
- Budgets and terminal outcomes are enforced.
- Side effects have durable identities and recovery states.
- Evaluation includes malicious-looking data and partial failures.

Reliable next reads

- [OWASP GenAI Security Project](#)
- [GitHub Copilot agent responsible use](#)
- [Azure AI architecture guidance](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.